An Approximate Fuzzy Knowledge Based Systems Reasoning Model for Situation and Threat Assessment

Author	Professor	Institution
OCdt Syed A.M.	Dr. Yawei Liang	Royal Military College Canada

This lab develops a compact Fuzzy Knowledge-Based System (FKBS) to support situational and threat assessment for a naval task group operating in a littoral environment.

In the process of situation and threat assessment, the following 18 parameters are normally taken into account:

No.	Evidence	Details
01	Origin	Country of origin. Higher threat if from a hostile nation. (`H` = Hostile, `N` = Neutral, `F` = Friendly)
02	IFF Mode	Identification Friend or Foe mode: 1–2 (military), 3/A & C (civilian), 4 (encrypted/military).
03	Intel	Additional intelligence indicators or reports that may increase threat.
04	Air Route	Whether the contact is inside a commercial air corridor (`ON`) or not (`OFF`).
05	**Altitude**	**Altitude of the contact in feet.**
06	ESM	Electronic Support Measures (signals/intercepts indicating intent or capability).
07	**Speed**	**Speed in knots (nautical miles per hour).**
08	**CAP**	**Closest Point of Approach — how close the contact will pass the own ship (nm).**
09	Feet	Terrain type beneath the contact: `Wet` = over water, `Dry` = over land.
10	Number	Number of contacts in the formation.
11	Coord Act	Coordinated act — whether the contact is coordinating actions with others.
12	Manvr	Manoeuvre — the way the contact is flying (evasive, attack profile, loitering, etc.).
13	Wings	Whether the contact carries ordnance under its wings.
14	Heading	Bearing relative to the task group: `Close` = flying toward TG, `Away` = flying away.

No.	Evidence	Details
15	**Range**	**Distance between the contact and the task group (nautical miles).**
16	Own Supt	Own support — proximity of friendly ships and/or aircraft.
17	Vis	Visibility (distance) and prevailing environmental conditions.
18	Wpn Envl	Weapon envelope — whether the contact is within effective weapon release ranges (R-min to R-max).

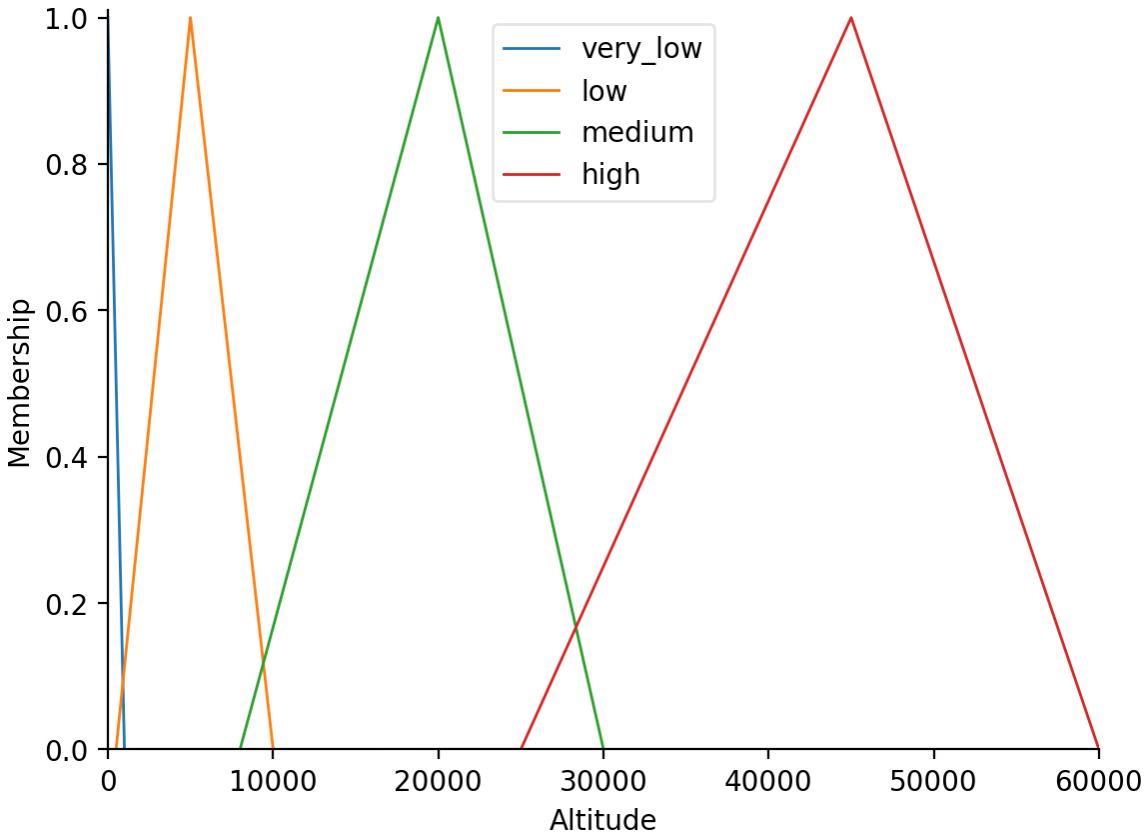
The exercise focuses on modelling uncertainty in four key numeric evidences (altitude, speed, closest point of approach, and range), designing membership functions, encoding human expert rules, and producing a continuous threat rating for each contact to aid operator decision-making.

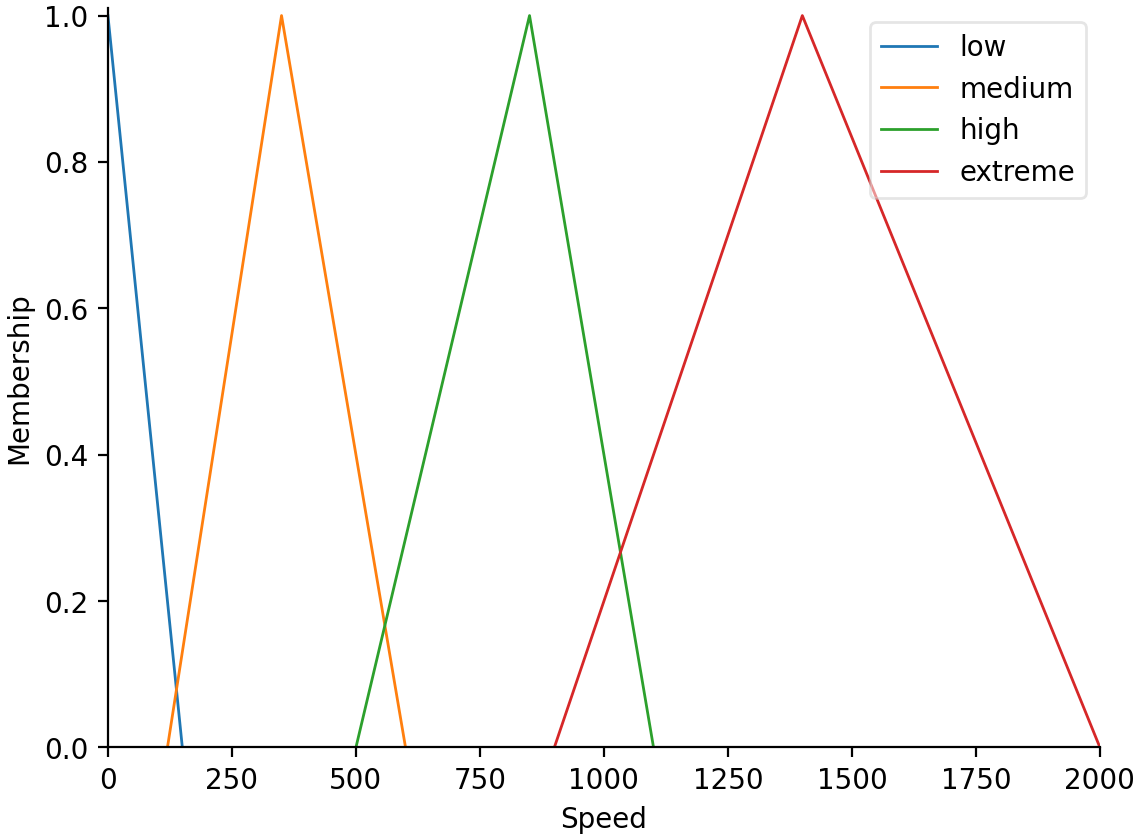
In [ ]:
```
%matplotlib inline
%config InlineBackend.figure_format = 'retina'
import matplotlib.pyplot as plt
import numpy as np
import skfuzzy as fuzz
import pandas as pd
from sklearn.impute import KNNImputer
from skfuzzy import control as ctrl
```

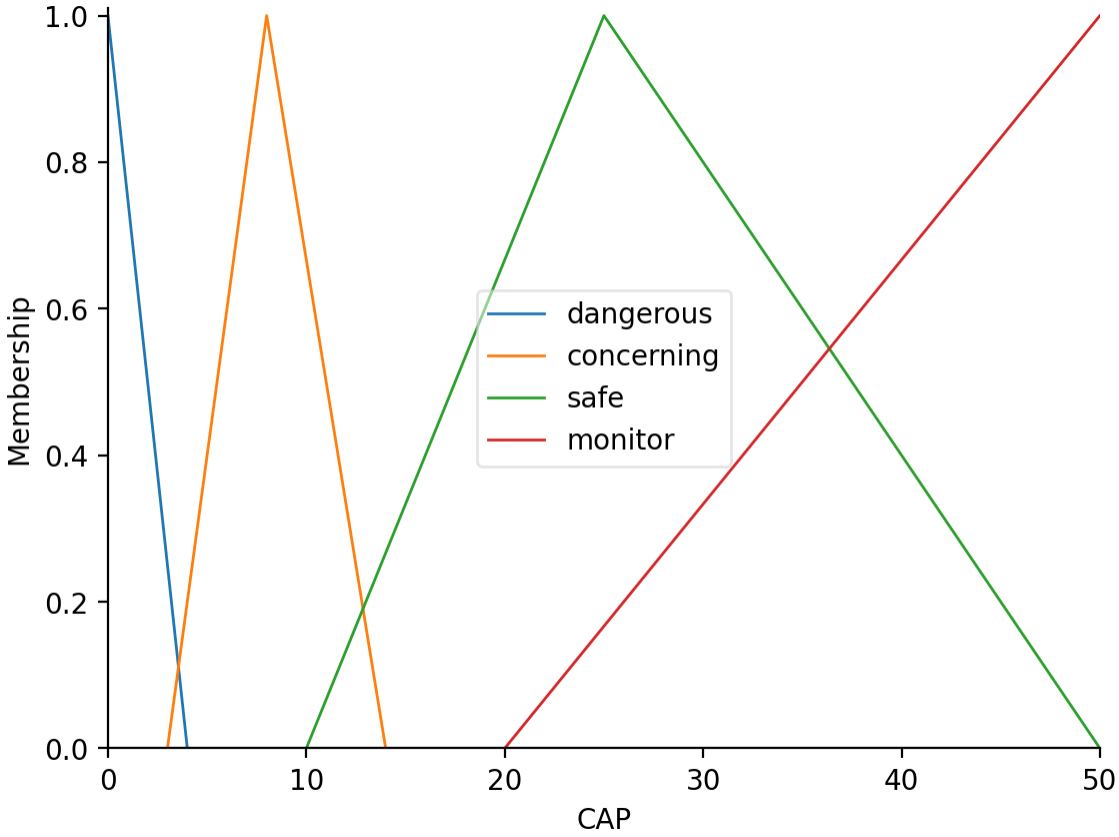
Membership Functions - Input

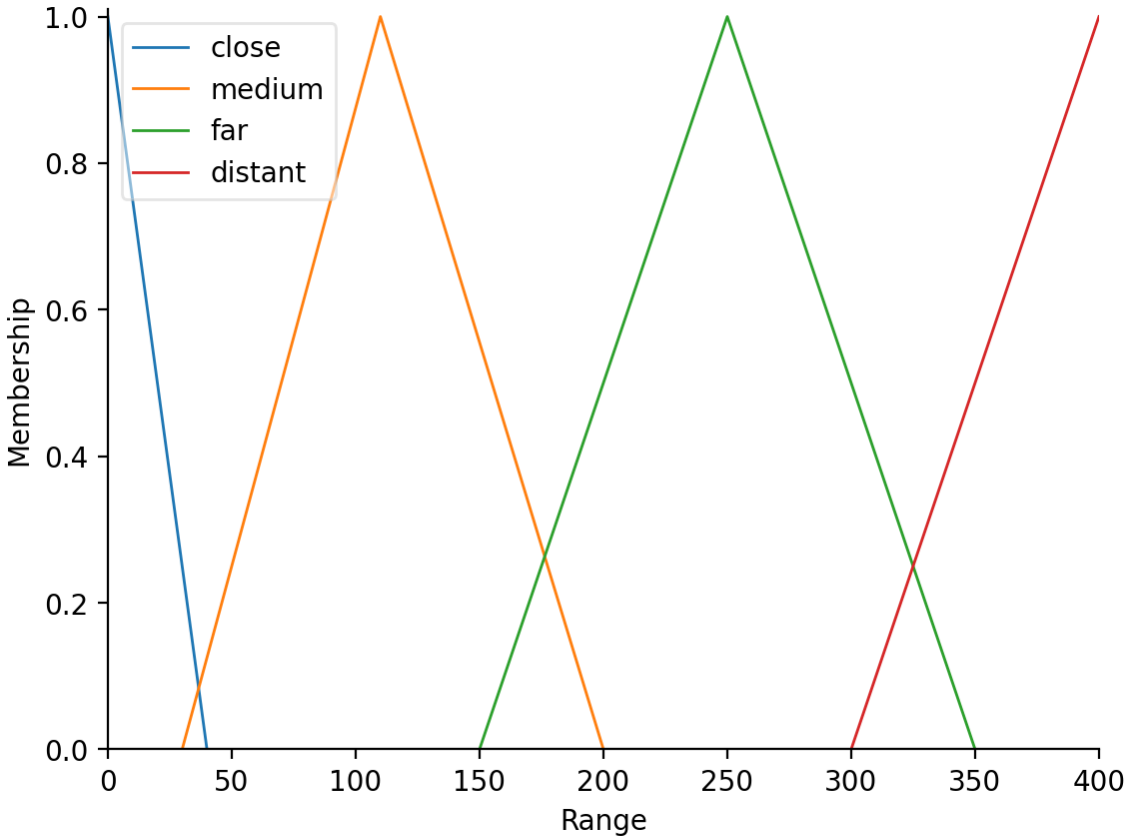
In [ ]:
```
# Altitude in feet (using generalized bell MFs)
Altitude = ctrl.Antecedent(np.arange(0, 60001, 1), "Altitude")
Altitude["very_low"] = fuzz.trimf(Altitude.universe, [0, 0, 1000])
Altitude["low"] = fuzz.trimf(Altitude.universe, [500, 5000, 10000])
Altitude["medium"] = fuzz.trimf(Altitude.universe, [8000, 20000, 30000])
Altitude["high"] = fuzz.trimf(Altitude.universe, [25000, 45000, 60000])

# Speed in knots (0–2000)
Speed = ctrl.Antecedent(np.arange(0, 2001, 1), "Speed")
Speed["low"] = fuzz.trimf(Speed.universe, [0, 0, 150])  # drones/rotary very
Speed["medium"] = fuzz.trimf(Speed.universe, [120, 350, 600])  # transports/
Speed["high"] = fuzz.trimf(Speed.universe, [500, 850, 1100])  # fighters/bom
Speed["extreme"] = fuzz.trimf(
    Speed.universe, [900, 1400, 2000]
)  # missiles/hypersonic (narrower left edge)

# Closest Point of Approach (CPA) in nm (0–50)
CAP = ctrl.Antecedent(np.arange(0, 51, 1), "CAP")
CAP["dangerous"] = fuzz.trimf(CAP.universe, [0, 0, 4])  # < ~4nm very danger
CAP["concerning"] = fuzz.trimf(CAP.universe, [3, 8, 14])
CAP["safe"] = fuzz.trimf(CAP.universe, [10, 25, 50])
CAP["monitor"] = fuzz.trimf(CAP.universe, [20, 50, 80])

# Range in nm (0–400)
Range = ctrl.Antecedent(np.arange(0, 401, 1), "Range")
Range["close"] = fuzz.trimf(Range.universe, [0, 0, 40])  # immediate <~40nm
```

```
Range["medium"] = fuzz.trimf(Range.universe, [30, 110, 200])
Range["far"] = fuzz.trimf(Range.universe, [150, 250, 350])
Range["distant"] = fuzz.trimf(Range.universe, [300, 400, 600])


Altitude.view()
Speed.view()
CAP.view()
Range.view()
plt.show()
```




low
medium
high
extreme
Membership
Speed
0.0
0.2
0.4
0.6
0.8
1.0
0
250
500
750
1000
1250
1500
1750
2000

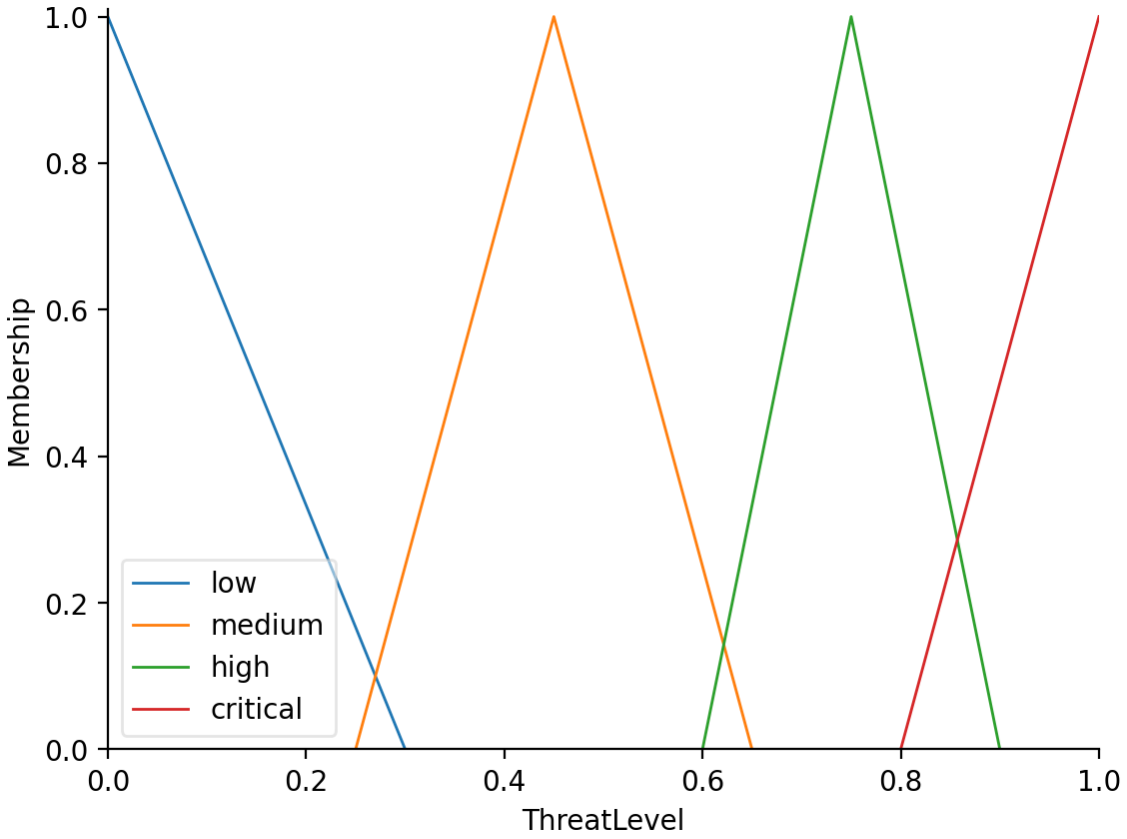

dangerous
concerning
safe
monitor
Membership
CAP
0.0
0.2
0.4
0.6
0.8
1.0
0
10
20
30
40
50



Output

In [226…

```
ThreatLevel = ctrl.Consequent(np.arange(0, 1.01, 0.01), "ThreatLevel")
ThreatLevel["low"] = fuzz.trimf(ThreatLevel.universe, [0, 0, 0.3])
ThreatLevel["medium"] = fuzz.trimf(ThreatLevel.universe, [0.25, 0.45, 0.65])
ThreatLevel["high"] = fuzz.trimf(ThreatLevel.universe, [0.6, 0.75, 0.9])
ThreatLevel["critical"] = fuzz.trimf(ThreatLevel.universe, [0.8, 1, 1])

ThreatLevel.view()
plt.show()
```



Inference Rules

The fuzzy rule base is designed to capture **realistic air and missile threats** to a task group.
It considers **altitude, speed, range, and closest point of approach (CPA)**.

Key Threat Categories include:

- Sea-skimming missiles **(Critical Threat)**
  - Very low altitude, extreme speed, close range, dangerous CPA.
- Bombers / Strike Aircraft **(High Threat)**
  - Medium altitude, high speed, medium range.
- Civilian Airliners **(Low Threat)**
  - Medium altitude, medium speed, distant range.
- Recon / High-Altitude Craft **(Medium Threat)**
  - High altitude, high speed, far range.

In [ ]:
```
rules = []

# Missiles (sea-skimmer or hypersonic)
rules.append(
    ctrl.Rule(
        antecedent=Altitude["very_low"] & Speed["extreme"] & CAP["dangerous"
        consequent=ThreatLevel["critical"],
```

```
    )
)
rules.append(
    ctrl.Rule(
        antecedent=Speed["extreme"] & Range["medium"],
        consequent=ThreatLevel["critical"],
    )
)
rules.append(
    ctrl.Rule(
        antecedent=Speed["extreme"] & Range["distant"], consequent=ThreatLev
    )
)

# Strike fighters / bombers
rules.append(
    ctrl.Rule(
        antecedent=Altitude["low"] & Speed["high"] & CAP["dangerous"],
        consequent=ThreatLevel["high"],
    )
)
rules.append(
    ctrl.Rule(
        antecedent=Altitude["medium"]
        & Speed["high"]
        & Range["medium"]
        & CAP["concerning"],
        consequent=ThreatLevel["high"],
    )
)

# Recon / high flyers
rules.append(
    ctrl.Rule(
        antecedent=Altitude["high"] & Speed["high"] & Range["far"],
        consequent=ThreatLevel["medium"],
    )
)
rules.append(
    ctrl.Rule(
        antecedent=Altitude["high"] & Speed["medium"] & Range["distant"],
        consequent=ThreatLevel["low"],
    )
)

# Civilian profiles
rules.append(
    ctrl.Rule(
        antecedent=Altitude["medium"]
        & Speed["medium"]
        & Range["distant"]
        & CAP["safe"],
        consequent=ThreatLevel["low"],
    )
)
```

```
# Drones / rotary
rules.append(
    ctrl.Rule(
        antecedent=Altitude["low"] & Speed["low"] & Range["close"],
        consequent=ThreatLevel["medium"],
    )
)

# Overrides to force certain obvious values for certain conditions
rules.append(
    ctrl.Rule(
        antecedent=Range["close"] & CAP["dangerous"], consequent=ThreatLevel
    )
)
rules.append(
    ctrl.Rule(
        antecedent=Range["medium"] & CAP["concerning"], consequent=ThreatLev
    )
)
rules.append(
    ctrl.Rule(antecedent=Range["far"] & CAP["safe"], consequent=ThreatLevel[
)

rules.append(
    ctrl.Rule(
        Altitude["low"] & Speed["low"] & Range["close"] & CAP["concerning"],
        ThreatLevel["medium"],
    )
)

# if drone is extremely close + dangerous CPA, escalate to HIGH (local safet
rules.append(
    ctrl.Rule(
        Altitude["low"] & Speed["low"] & Range["close"] & CAP["dangerous"],
        ThreatLevel["high"],
    )
)

# --- Hypersonic/missile rules: extreme speed escalates to CRITICAL as range
rules.append(
    ctrl.Rule(
        Speed["extreme"] & (Range["medium"] | Range["close"]) & CAP["concern
        ThreatLevel["critical"],
    )
)

# Hypersonic at long range is high (early warning) but not ignored
rules.append(ctrl.Rule(Speed["extreme"] & Range["far"], ThreatLevel["high"])
```

In [228…

```
"""
    Control System
"""
ThreatLevelSimulation = ctrl.ControlSystemSimulation(ctrl.ControlSystem(rule
```

Testing the FKBS Model

In [ ]:
```
type feet = int
type knots = int
type nautical_miles = int
type threat_rating = int


def SimulateThreatRating(
    altitude: feet,
    speed: knots,
    closest_approach_point: nautical_miles,
    range: nautical_miles,
) -> threat_rating:
    ThreatLevelSimulation.input["Altitude"] = altitude
    ThreatLevelSimulation.input["Speed"] = speed
    ThreatLevelSimulation.input["CAP"] = closest_approach_point
    ThreatLevelSimulation.input["Range"] = range
    ThreatLevelSimulation.compute()

    return ThreatLevelSimulation.output["ThreatLevel"]


def get_threat_label_fuzzy(value):
    memberships = {
        "LOW": fuzz.interp_membership(
            ThreatLevel.universe, ThreatLevel["low"].mf, value
        ),
        "MEDIUM": fuzz.interp_membership(
            ThreatLevel.universe, ThreatLevel["medium"].mf, value
        ),
        "HIGH": fuzz.interp_membership(
            ThreatLevel.universe, ThreatLevel["high"].mf, value
        ),
        "CRITICAL": fuzz.interp_membership(
            ThreatLevel.universe, ThreatLevel["critical"].mf, value
        ),
    }
    return max(memberships, key=memberships.get)  # pick label with max memb
```

Simulated Realtime Examples

In [ ]:
```
def ShowSimulatedThreatRating(name: str, csv_name: str):
    # Read data
    track_data = pd.read_csv(csv_name)

    # Safe simulation: return NaN on error or non-finite result
    def safe_simulate_nan(row):
        try:
            v = SimulateThreatRating(
                row["Altitude_ft"], row["Speed_kts"], row["CAP_nm"], row["Ra
            )
            return float(v) if (v is not None and np.isfinite(v)) else np.na
        except Exception:
```

```
        return np.nan

track_data["ThreatRating"] = track_data.apply(safe_simulate_nan, axis=1)

# Numeric columns used for imputation
numeric_cols = ["Altitude_ft", "Speed_kts", "CAP_nm", "Range_nm", "Threa
missing_cols = [c for c in numeric_cols if c not in track_data.columns]
if missing_cols:
    raise KeyError(f"Missing columns in {csv_name}: {missing_cols}")

df_num = track_data[numeric_cols].astype(float).copy()
n_samples = len(df_num)

# Fill entirely-NaN columns with 0.0
all_nan_cols = df_num.columns[df_num.isna().all()].tolist()
if all_nan_cols:
    df_num[all_nan_cols] = df_num[all_nan_cols].fillna(0.0)

# Impute remaining NaNs: median fill for tiny sample, else KNNImputer
if df_num.isna().any().any():
    if n_samples < 2:
        df_num = df_num.fillna(df_num.median().fillna(0.0))
    else:
        imputer = KNNImputer(n_neighbors=min(3, max(1, n_samples - 1)))
        arr_imputed = imputer.fit_transform(df_num.values)
        df_num = pd.DataFrame(arr_imputed, columns=numeric_cols, index=d

# Ensure ThreatRating numeric and clipped to [0,1]
track_data["ThreatRating"] = (
    pd.to_numeric(df_num["ThreatRating"], errors="coerce")
    .fillna(0.0)
    .clip(0.0, 1.0)
)

# Map numeric -> label (try fuzzy mapper, else threshold fallback)
def map_label(value):
    try:
        return get_threat_label_fuzzy(value, ThreatLevel)
    except Exception:
        if value <= 0.3:
            return "LOW"
        if value <= 0.6:
            return "MEDIUM"
        if value <= 0.85:
            return "HIGH"
        return "CRITICAL"

track_data["ThreatLevel"] = track_data["ThreatRating"].apply(map_label)

def safe_norm(col):
    mx = track_data[col].max()
    return track_data[col] / (mx if (mx and np.isfinite(mx)) else 1.0)

alt_norm = safe_norm("Altitude_ft")
speed_norm = safe_norm("Speed_kts")
cap_norm = safe_norm("CAP_nm")
```

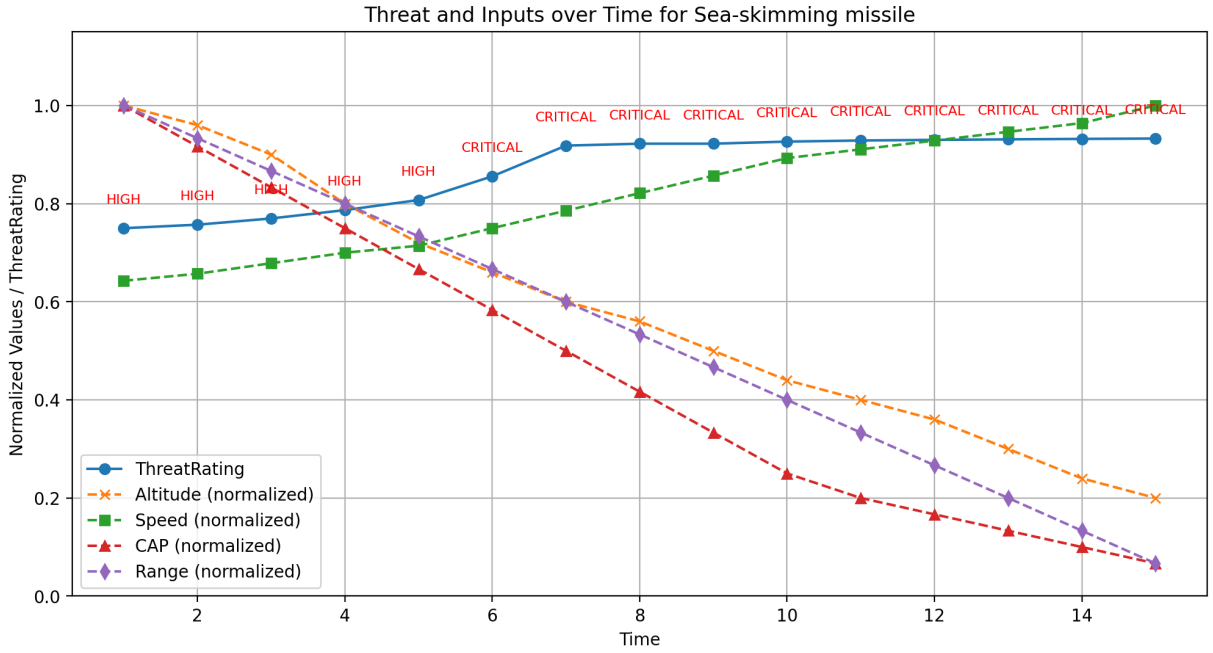
```
    range_norm = safe_norm("Range_nm")

    plt.figure(figsize=(12, 6))
    plt.plot(
        track_data["time"],
        track_data["ThreatRating"],
        marker="o",
        linestyle="-",
        label="ThreatRating",
    )
    plt.plot(
        track_data["time"],
        alt_norm,
        marker="x",
        linestyle="--",
        label="Altitude (normalized)",
    )
    plt.plot(
        track_data["time"],
        speed_norm,
        marker="s",
        linestyle="--",
        label="Speed (normalized)",
    )
    plt.plot(
        track_data["time"],
        cap_norm,
        marker="^",
        linestyle="--",
        label="CAP (normalized)",
    )
    plt.plot(
        track_data["time"],
        range_norm,
        marker="d",
        linestyle="--",
        label="Range (normalized)",
    )

    for t, val, label in zip(
        track_data["time"], track_data["ThreatRating"], track_data["ThreatLe
    ):
        y = float(val) + 0.05
        if y > 1.05:
            y = float(val) - 0.06
        plt.text(t, y, label, ha="center", fontsize=8, color="red")

    plt.title(f"Threat and Inputs over Time for {name}")
    plt.xlabel("Time")
    plt.ylabel("Normalized Values / ThreatRating")
    plt.ylim(0, 1.15)
    plt.grid(True)
    plt.legend()
    plt.show()

    return track_data
```

Sea-skimming missile: Critical

Very low, extreme speed, tiny CPA, closes fast (hardest threat to detect/react).

In [231…

```
ShowSimulatedThreatRating("Sea-skimming missile", "examples/sea_skimmer.csv"
```



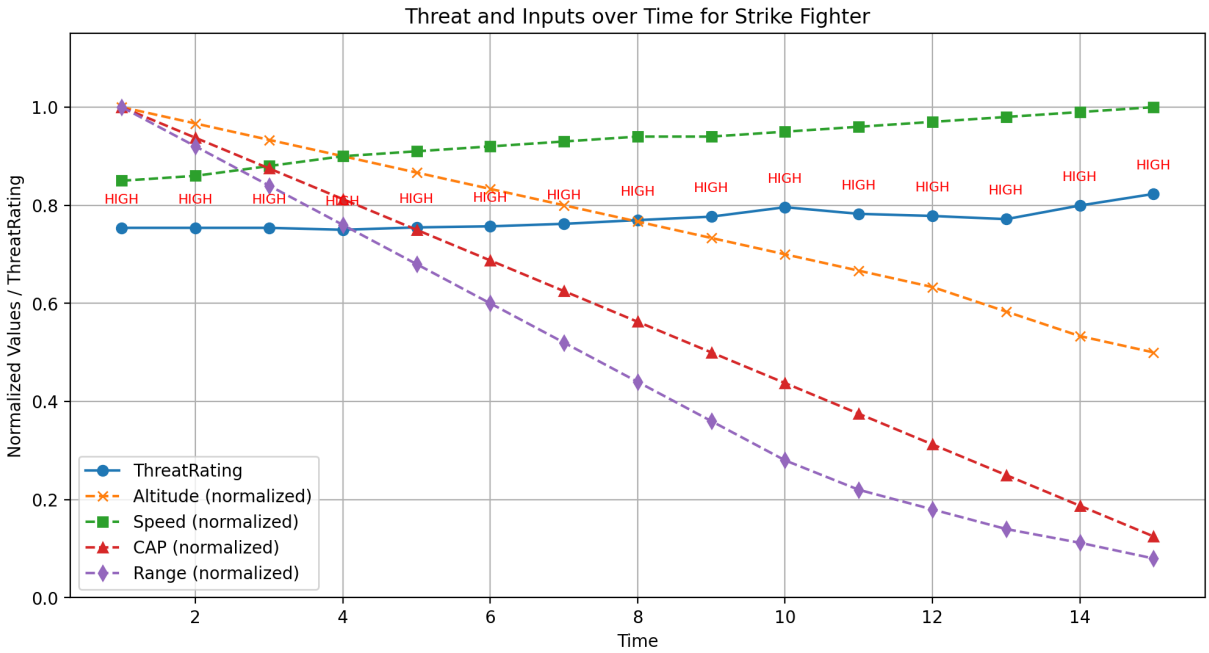
Out[231…

	time	Altitude_ft	Speed_kts	CAP_nm	Range_nm	ThreatRating	ThreatLevel
0	1	50	900	6.0	150	0.750000	HIGH
1	2	48	920	5.5	140	0.757107	HIGH
2	3	45	950	5.0	130	0.769844	HIGH
3	4	40	980	4.5	120	0.786854	HIGH
4	5	36	1000	4.0	110	0.807403	HIGH
5	6	33	1050	3.5	100	0.855760	CRITICAL
6	7	30	1100	3.0	90	0.918333	CRITICAL
7	8	28	1150	2.5	80	0.922222	CRITICAL
8	9	25	1200	2.0	70	0.922222	CRITICAL
9	10	22	1250	1.5	60	0.926515	CRITICAL
10	11	20	1275	1.2	50	0.928718	CRITICAL
11	12	18	1300	1.0	40	0.930000	CRITICAL
12	13	15	1325	0.8	30	0.931111	CRITICAL
13	14	12	1350	0.6	20	0.932029	CRITICAL
14	15	10	1400	0.4	10	0.932727	CRITICAL

Strike Fighter: High

Low altitude, high speed, inbound, within weapons release range.

In [232…
```
ShowSimulatedThreatRating("Strike Fighter", "examples/strike_fighter.csv")
```



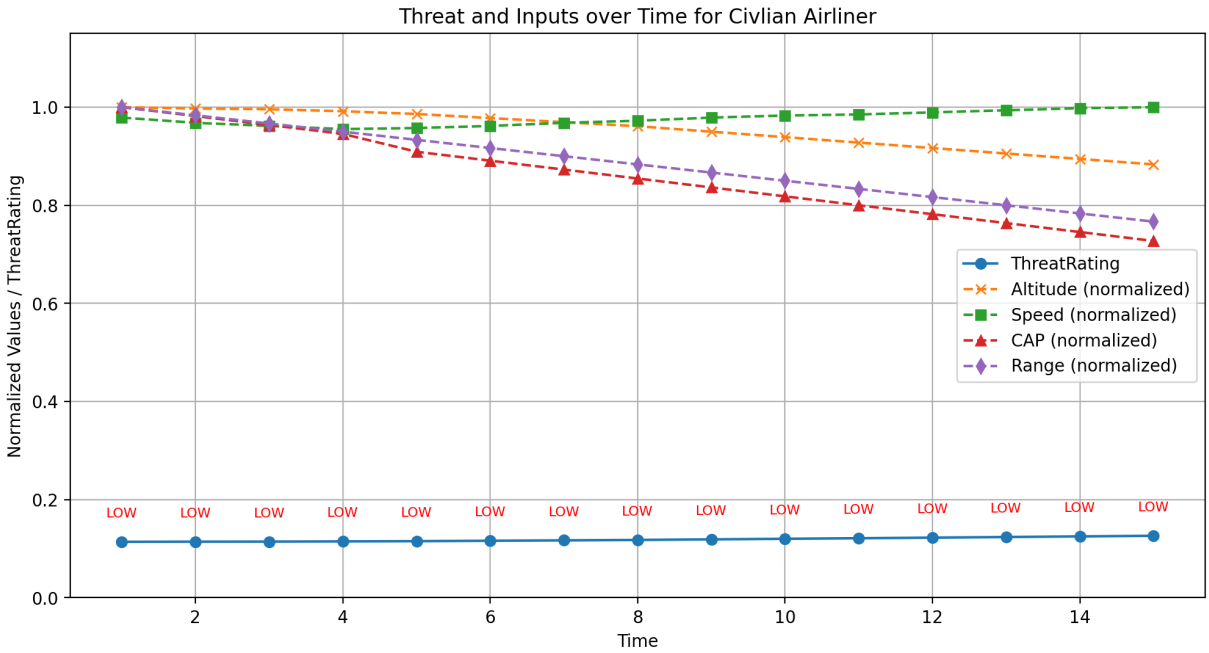
Out[232…

	time	Altitude_ft	Speed_kts	CAP_nm	Range_nm	ThreatRating	ThreatLevel
0	1	6000	850	8.0	250	0.753987	HIGH
1	2	5800	860	7.5	230	0.753987	HIGH
2	3	5600	880	7.0	210	0.753987	HIGH
3	4	5400	900	6.5	190	0.750000	HIGH
4	5	5200	910	6.0	170	0.754854	HIGH
5	6	5000	920	5.5	150	0.757107	HIGH
6	7	4800	930	5.0	130	0.762160	HIGH
7	8	4600	940	4.5	110	0.769726	HIGH
8	9	4400	940	4.0	90	0.776823	HIGH
9	10	4200	950	3.5	70	0.796058	HIGH
10	11	4000	960	3.0	55	0.782538	HIGH
11	12	3800	970	2.5	45	0.778193	HIGH
12	13	3500	980	2.0	35	0.771708	HIGH
13	14	3200	990	1.5	28	0.799502	HIGH
14	15	3000	1000	1.0	20	0.823114	HIGH

Civilian Airliner: Low

Medium altitude, medium speed, large CPA, outbound/distant, non-threat profile.

In [233…
```
ShowSimulatedThreatRating("Civlian Airliner", "examples/airliner.csv")
```



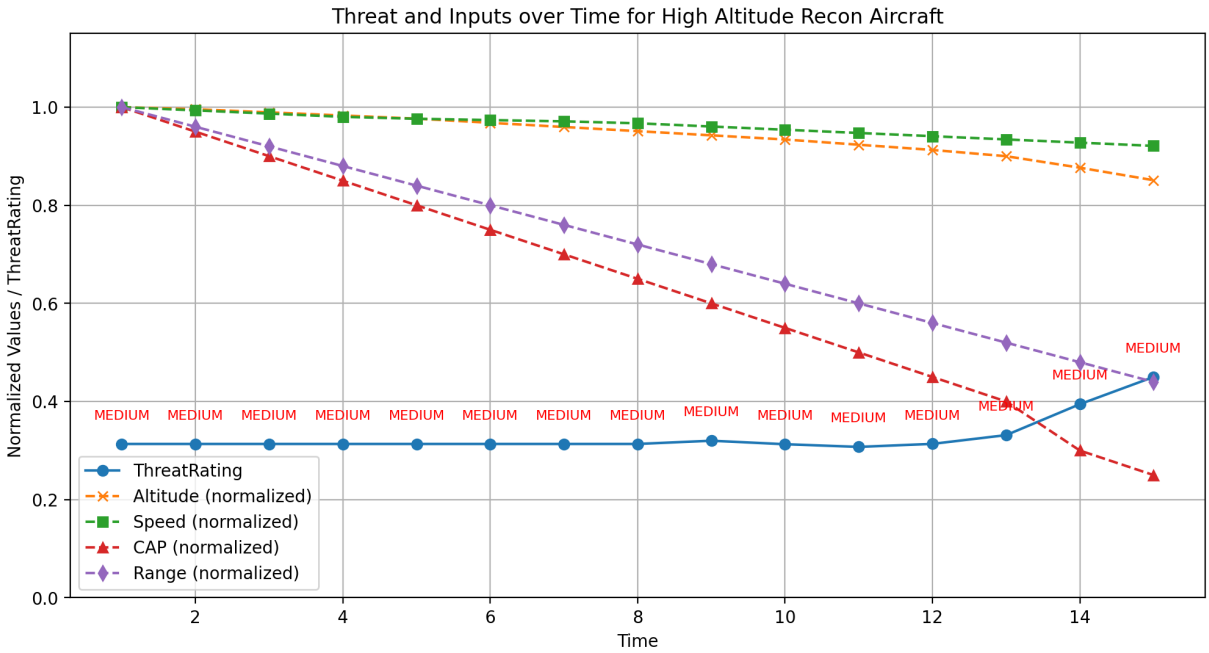
Out[233…

	time	Altitude_ft	Speed_kts	CAP_nm	Range_nm	ThreatRating	ThreatLevel
0	1	36000	460	55	600	0.113966	LOW
1	2	35900	455	54	590	0.114229	LOW
2	3	35850	452	53	580	0.114361	LOW
3	4	35700	449	52	570	0.114759	LOW
4	5	35500	450	50	560	0.115297	LOW
5	6	35200	452	49	550	0.116114	LOW
6	7	34900	455	48	540	0.116945	LOW
7	8	34600	457	47	530	0.117789	LOW
8	9	34200	460	46	520	0.118935	LOW
9	10	33800	462	45	510	0.120103	LOW
10	11	33400	463	44	500	0.121291	LOW
11	12	33000	465	43	490	0.122500	LOW
12	13	32600	467	42	480	0.123728	LOW
13	14	32200	469	41	470	0.124976	LOW
14	15	31800	470	40	460	0.126241	LOW

High Altitude Recon Aircraft: Medium

High altitude, high speed, but distant with larger CPA → monitor, not immediate.

In [234…
```
ShowSimulatedThreatRating("High Altitude Recon Aircraft", "examples/recon_hi
```



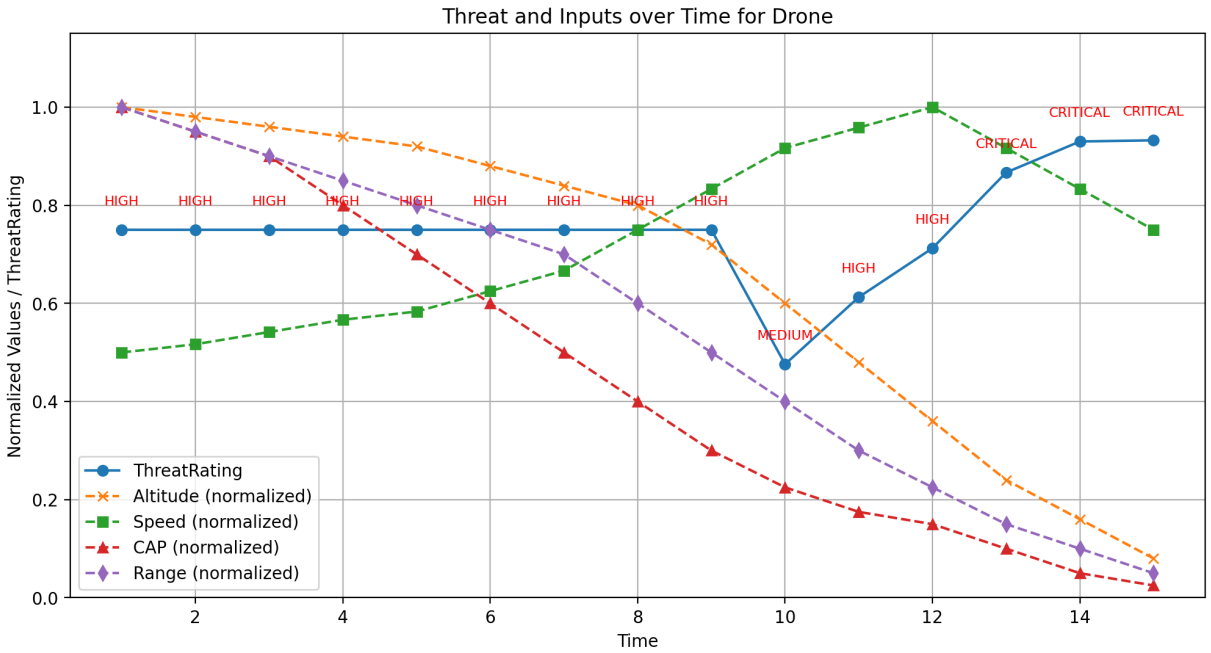
Out[234…

	time	Altitude_ft	Speed_kts	CAP_nm	Range_nm	ThreatRating	ThreatLevel
0	1	47000	760	40	500	0.313467	MEDIUM
1	2	46800	755	38	480	0.313467	MEDIUM
2	3	46500	750	36	460	0.313467	MEDIUM
3	4	46200	745	34	440	0.313467	MEDIUM
4	5	45900	742	32	420	0.313467	MEDIUM
5	6	45500	740	30	400	0.313467	MEDIUM
6	7	45100	738	28	380	0.313467	MEDIUM
7	8	44700	735	26	360	0.313467	MEDIUM
8	9	44300	730	24	340	0.320026	MEDIUM
9	10	43900	725	22	320	0.312907	MEDIUM
10	11	43400	720	20	300	0.307468	MEDIUM
11	12	42900	715	18	280	0.313607	MEDIUM
12	13	42300	710	16	260	0.331460	MEDIUM
13	14	41200	705	12	240	0.394527	MEDIUM
14	15	40000	700	10	220	0.450000	MEDIUM

Drone: Medium

Low altitude, slow, close proximity → not lethal but proximity = concern.

In [235…
```
ShowSimulatedThreatRating("Drone", "examples/drone.csv")
```



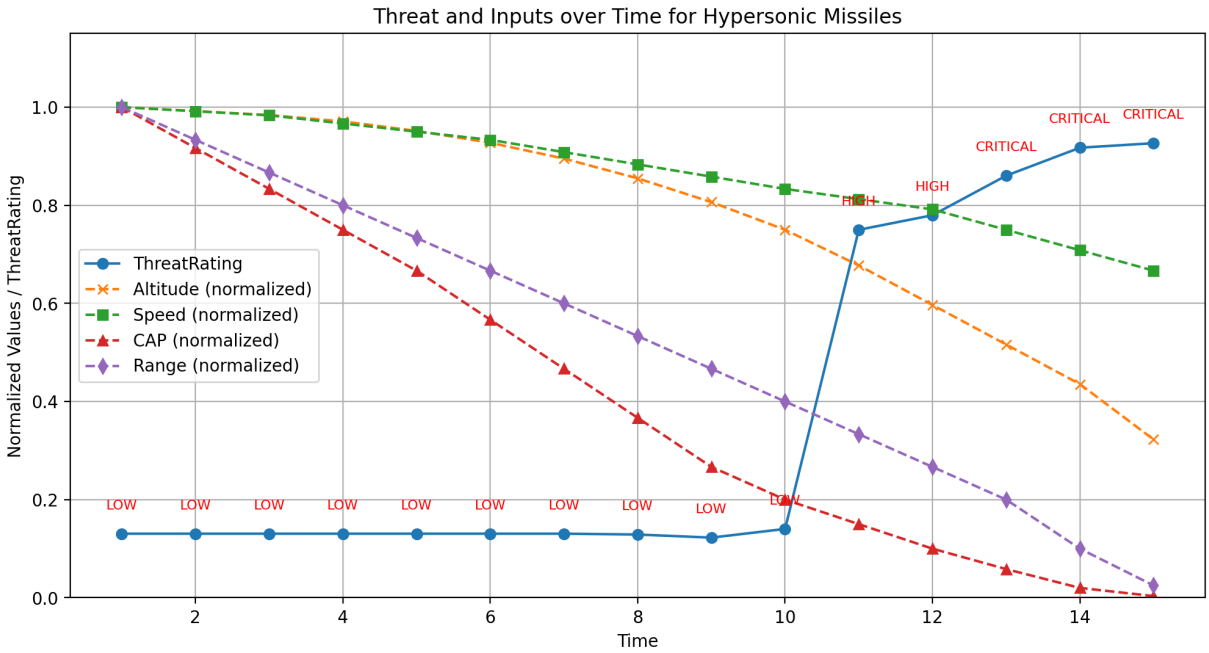
Out[235…

	time	Altitude_ft	Speed_kts	CAP_nm	Range_nm	ThreatRating	ThreatLevel
0	1	2500	60	20.0	80	0.750000	HIGH
1	2	2450	62	19.0	76	0.750000	HIGH
2	3	2400	65	18.0	72	0.750000	HIGH
3	4	2350	68	16.0	68	0.750000	HIGH
4	5	2300	70	14.0	64	0.750000	HIGH
5	6	2200	75	12.0	60	0.750000	HIGH
6	7	2100	80	10.0	56	0.750000	HIGH
7	8	2000	90	8.0	48	0.750000	HIGH
8	9	1800	100	6.0	40	0.750000	HIGH
9	10	1500	110	4.5	32	0.475978	MEDIUM
10	11	1200	115	3.5	24	0.612680	HIGH
11	12	900	120	3.0	18	0.712460	HIGH
12	13	600	110	2.0	12	0.866828	CRITICAL
13	14	400	100	1.0	8	0.930000	CRITICAL
14	15	200	90	0.5	4	0.932407	CRITICAL

Hypersonic Missiles: Critical

Very high altitude, extreme speed, rapid range reduction → catastrophic threat.

In [236…
```
ShowSimulatedThreatRating("Hypersonic Missiles", "examples/hypersonic.csv")
```



Out[236…

	time	Altitude_ft	Speed_kts	CAP_nm	Range_nm	ThreatRating	ThreatLevel
0	1	62000	2400	60.0	600	0.130521	LOW
1	2	61500	2380	55.0	560	0.130521	LOW
2	3	61000	2360	50.0	520	0.130521	LOW
3	4	60200	2320	45.0	480	0.130521	LOW
4	5	59000	2280	40.0	440	0.130521	LOW
5	6	57500	2240	34.0	400	0.130521	LOW
6	7	55500	2180	28.0	360	0.130521	LOW
7	8	53000	2120	22.0	320	0.128824	LOW
8	9	50000	2060	16.0	280	0.122500	LOW
9	10	46500	2000	12.0	240	0.140238	LOW
10	11	42000	1950	9.0	200	0.750000	HIGH
11	12	37000	1900	6.0	160	0.779696	HIGH
12	13	32000	1800	3.5	120	0.860321	CRITICAL
13	14	27000	1700	1.2	60	0.917308	CRITICAL
14	15	20000	1600	0.2	15	0.926515	CRITICAL

Conclusion

In this lab, we built a compact, interpretable FKBS using Altitude/Speed/CAP/Range membership functions and simple expert rules (with proximity overrides). After tuning and NaN handling it reliably separates benign from urgent contacts and is ready for further validation. Using only four evidences, this FKBS still produces reasonable threat assessments, though a full 18-evidence model would be more comprehensive.